

Гибкие функции: работа с произвольным числом аргументов через `*args` и `**kwargs`

1. Что такое `*` и `**`?

Прежде чем переходить к терминам, важно понять саму суть символов:

- Одна звездочка (`*`) - это оператор распаковки (или упаковки) последовательности. Она говорит Python: «Собери все оставшиеся позиционные аргументы в один кортеж (tuple)».
- Две звездочки (`**`) - это оператор распаковки словаря. Она говорит Python: «Собери все именованные аргументы (пары ключ=значение) в один словарь (dict)».

2. Позиционные аргументы: `*args`

Название `args` - это сокращение от *arguments*. Вы можете назвать это как угодно (например, `*numbers`), но `*args` - это стандарт среди программистов. Зачем это нужно? Когда вы хотите передать в функцию 2, 5 или 100 значений, не создавая для каждого отдельный параметр.

Шаблон:

```
def имя_функции(*args):  
    # args - это кортеж  
    for item in args:  
        print(item)
```

Пример: Функция для суммирования всех чисел

```
def sum_all(*numbers):  
    total = sum(numbers)  
    return total  
  
print(sum_all(1, 2, 3))          # Вернет 6  
print(sum_all(10, 20, 30, 40)) # Вернет 100
```

3. Именованные аргументы: `**kwargs`

Название `kwargs` - это сокращение от *keyword arguments*.

Зачем это нужно? Когда вы хотите передать данные с пояснениями (ключами), но не знаете заранее, какие именно характеристики будут переданы.

Шаблон:

```
def имя_функции(**kwargs):  
    # kwargs - это словарь  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")
```

Пример: Профиль пользователя

```
def build_profile(name, **info):  
    print(f"Пользователь: {name}")  
    for key, value in info.items():  
        print(f"- {key}: {value}")  
  
build_profile("Алихан", city="Astana", age=25,  
job="Python Developer")
```

4. Порядок аргументов (Важно!)

Если вы используете всё сразу, Python требует соблюдать строгий порядок в скобках:

1. Обычные аргументы.
2. *args.
3. **kwargs.

```
def complex_function(a, b, *args, **kwargs):  
    pass
```

Примеры использования на практике

1. Принт-функции: Вспомните `print(1, 2, 3, sep="-")`. Здесь 1, 2, 3 уходят в *args, а sep - это часть **kwargs.
2. Декораторы и обертки: Когда одна функция вызывает другую и должна передать ей абсолютно все полученные данные без изменений.
3. Настройки (Config): Когда функция принимает десятки необязательных параметров оформления (цвета, шрифты, отступы).



Задачи для закрепления

Задача 1: «Бесконечный привет»

Напишите функцию `greet_all(*names)`, которая принимает любое количество имен и для каждого выводит: «Привет, [имя]!».

Задача 2: «Умножитель»

Создайте функцию `multiply_all(*numbers)`. Она должна принимать произвольное количество чисел и возвращать результат их перемножения. Если чисел нет, пусть возвращает 1.

Задача 3: «Описание товара»

Напишите функцию `describe_item(title, **features)`. Первый аргумент - название товара. Остальные (`**kwargs`) - его характеристики. *Пример вызова:* `describe_item("Laptop", brand="Apple", ram="16GB", cpu="M2")`. Функция должна напечатать название и перечислить все характеристики.

Задача 4: «Самый длинный список»

Напишите функцию `count_args(*args)`, которая просто возвращает количество переданных в неё аргументов.

Задача 5: «Сборщик чека»

Создайте функцию `print_bill(**items)`. Она принимает названия продуктов и их цены. Функция должна вывести красивый список: «[Продукт] - [Цена] тг», а в конце напечатать общую сумму.



Итог:

- `*args` превращает аргументы в кортеж.
- `**kwargs` превращает аргументы в словарь.
- Это позволяет вашим функциям быть «резиновыми» и подстраиваться под любые данные.